

bugbounty-agent

מסמך אפיון פרויקט מקיף

מישור בקרה בטוח לאוטומציה של Bug-Bounty / Pentest מורשה

תורגם מהמקור באנגלית

סטטוס: אפיון והיערכות הושלמו • המימוש טרם החל

היקף: כל ההחלטות שהתקבלו בסשן התכנון

קהל יעד: הארכיטקט הסוקר וכל מהנדס/סוכן שיממש את המערכת

תוכן העניינים

1. תקציר מנהלים
2. מטרות ולא-מטרות (Goals & Non-goals)
3. עקרונות בטיחות בלתי-ניתנים למשא-ומתן (Safety Invariants)
4. ארכיטקטורה וגבולות אמון (Trust Boundaries)
5. מודל האיום - מה שינה מעבר ה-Red-team
6. לולאת הבקרה האֶגְנֵטִית (Control Loop)
7. שער ההיקף (Scope-gate) - כיצד עוטפים כל קריאה לכלי
8. אישור אנושי בלולאה (Human-in-the-loop)
9. מאגר הממצאים ויומן הביקורת (Findings Store & Audit Log)
10. מצאי הכלים והערבויות הלא-הרסניות
11. קטלוג ה-Skills
12. תת-הסוכנים וחוזה ההעברה (Hand-off Contract)
13. מחסנית טכנולוגית (Tech Stack)
14. סכימת התצורה (Configuration Schema)
15. החלטות שנעלו ופסיקות
16. שאלות פתוחות וסיכונים שיוריים
17. מפת דרכים למימוש (Roadmap)
18. נספח: מילון מונחים (Glossary)

1. תקציר מנהלים

`bugbounty-agent` הוא מתזמר אגנטי (agentic orchestrator) לעבודת Bug-Bounty ובדיקות חדירה מורשות. הוא מניע **מכונת-מצבים דטרמיניסטית בת חמישה שלבים** - Recon → Enumeration → Vulnerability - discovery → Validation/triage → Reporting. **ה-LLM מסיק אך לעולם אינו פועל**. הפלט היחיד של ה-LLM הוא `ActionProposal` מאומת-סכימה (איזה כלי, אילו יעדים, ומדוע). ההצעה חוצה גבול-אמון יחיד אל **ליבה מהימנה (trusted kernel)** אשר מאמתת מחדש, באופן עצמאי, כל דבר מול `config/scope.yaml`. מאחר שההצעה אינה נושאת שום סמכות, מודל שעבר Prompt-injection או שהוזה יכול, במקרה הגרוע, רק לפלוט הצעה נוספת שהליבה תסרב לה.

ערך המערכת הוא **מישור הבקרה הבטוח** וההסקה שבין הרצות הכלים - לא הסריקה עצמה. היא מתזמרת כלים בשלים ומוכחים (`subfinder`, `amass`, `httpx`, `katana`, `gau`, `naabu`, `nuclei`, `ffuf`) במקום לממש מחדש סורקים.

2. מטרות ולא-מטרות

מטרות

- אוטומציה של מחזור החיים מ-Recon ועד דיווח, עבור תוכנית אחת, מורשית במפורש.
- להבטיח שכל פעולה הנוגעת ביעד היא, באופן מוכח, בהיקף, מוגבלת-קצב, מתועדת, ולפעולות אקטיביות - מאושרת אנושית. הכל נאכף בקוד, לא בהנחיות (prompts).
- לשרשר תוצאות בין השלבים דרך מאגר ממצאים מתמיד ובר-ביקורת.
- לשמור על נתיב הבטיחות הקריטי חופשי מכל קריאת LLM, כך שיהיה בר-בדיקה יחידתית ללא מודל.

לא-מטרות

- לא פלטפורמת סריקה המונית; מעורבות (engagement) אחת / תוכנית אחת לכל מאגר.
- לא מסגרת ניצול (exploitation) או חימוש; הוכחת-היתכנות (PoC) היא מינימלית ולא-מחומשת.
- לא מימוש-מחדש של סורקים; אנו עוטפים כלים קיימים.
- לא מערכת אוטונומית של "שגר-ושכח"; אדם מאשר כל צעד אקטיבי.

3. עקרונות בטיחות בלתי-ניתנים למשא-ומתן (SAF)

ארבעת העקרונות נאכפים בקוד, הם הדבר הראשון שכל סוכן קורא (`CLAUDE.md` §0), ואינם מתירים חריגים.

- **SAF-1 - שער היקף/הרשאה קשיח**. כל פעולה שעלולה לגעת ביעד מאומתת מול `scope.yaml` לפני שהיא רצה. השער הוא **כשל-סגור (fail-closed)**: out-of-scope תמיד גובר על in-scope (אפילו על התאמת תו-

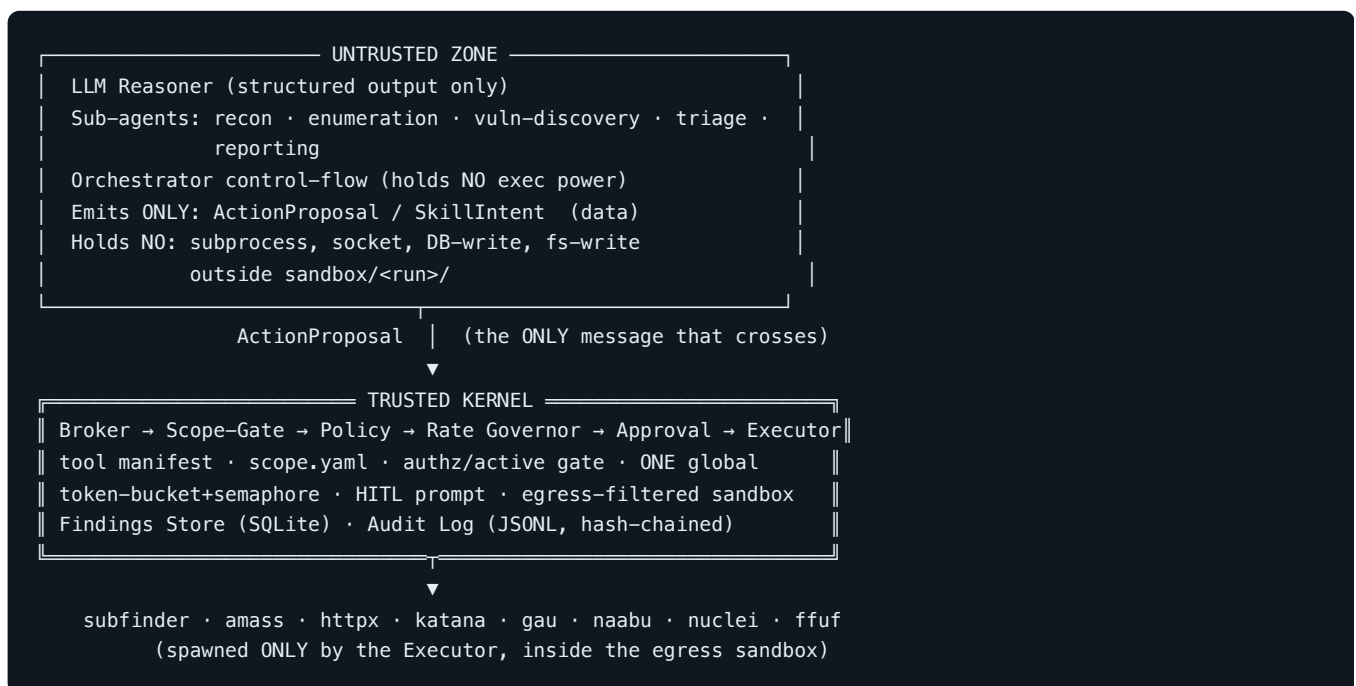
כללי; כל דבר לא-מאומת, דו-משמעי או בלתי-ניתן-לפירוק מסורב והריצה נעצרת (`DENY_HALT`). כל פעולה הנוגעת בתשתית היעד דורשת בנוסף `authorized == true` .

- **SAF-2 - אדם בלולאה לפני כל צעד אקטיבי.** OSINT פסיבי (אפס מנות ליעד) רשאי לרוץ תחילה. כל פעולה השמה מנה על תשתית היעד היא אקטיבית ודורשת, בזמן הרצה: `authorized == true` וגם `active_actions_allowed == true` וגם אסימון אישור אנושי חד-פעמי, לא-מרחיב, הכבול לאותה פקודה בדיוק. אין-TTY / פקיעה / EOF בערוץ האישור $\Leftarrow$ DENY.
- **SAF-3 - לא-הרסני כברירת מחדל.** אין DoS, אין ניצול מחומש, אין חילוץ נתוני-משתמש אמיתיים, אין קריאות out-of-band לצדדים שלישיים. בדיקות-אימות חוזרות הן קריאה-בלבד (GET/HEAD) כברירת מחדל; כל פועל משנה-מצב דורש אישור נפרד בדרג גבוה יותר. הממצאים מטופלים בגילוי אחראי (responsible disclosure) לבעל התוכנית.
- **SAF-4 - תיעוד מלא + הגבלת קצב + מאגר ממצאים מתמיד.** כל פעולה (מותרת או מסורבת) נכתבת ליומן ביקורת מוסף-בלבד ומשורשר-hash לפני שהיא רצה. כל העבודה היוצאת מקוצבת על-ידי מגביל-קצב גלובלי-יחיד לתהליך הנגזר מ- `scope.yaml` . מאגר ממצאים בבעלות הליבה הוא מקור-האמת המשרשר תוצאות בין השלבים.

כלל המוצא האחרון: אם אינך יכול לספק את כל הארבעה עבור פעולה - הפעולה אינה קורית. כאשר ההיקף או הבטיחות אינם ודאיים - עצור ושאל את האדם.

4. ארכיטקטורה וגבולות אמן

שני אזורים המופרדים בגבול יחיד של העברת-הודעות.



איור 1 - אזור לא-מהימן, ליבה מהימנה, והכלים המורצים אך ורק בתוך ה-sandbox.

- הגבול נחצה על-ידי סוג-הודעה יחיד בכל כיוון: `ActionProposal` פנימה, `ActionResult` מחוץ החוצה. אין הפניות לאובייקטים, `callables` או `file-handles` החוצים את הגבול.
- **שני קירות עצמאיים מתצורה אחת:** (1) בדיקות מדיניות על ה-`argv`/היעדים המורכבים, ו-(2) רשימת בקרת יציאה רשתית (`egress ACL`) הנוצרת מאותו `scope.yaml`. קיר 2 תופס את מה שבדיקת `argv` אינה יכולה – רקורסיה טבעית של כלים, הפניות (`redirects`), ו-`CNAME/DNS rebinding`. **כלים אקטיביים מסרבים לרוץ אלא אם מסכן היציאה אומת כפעיל באתחול.**
- כל שורת ממצא ו-`asset` נושאת שרשרת-מקור (`provenance`): `run_id → phase → agent → skill →`
`artifact → sandbox → audit_hash → approval → scope_decision → tool_invocation`

5. מודל האיום – מה שינה מעבר ה-Red-team

- העיצוב חוזק בשני סבבי סקירה יריבית (`adversarial`) שחיפשו עקיפות של שער-ההיקף ונתיבי פעולה אקטיבית לא-מאושרת. הממצאים שעיצבו את התכן הסופי:
- **הרחבת יעדים טבעית של כלים** (`subfinder` המזין תת-דומיינים שהתגלו, `katana` הזוחל אל מחוץ להיקף, `nuclei` העוקב אחרי הפניות) עלולה להגיע למארחים שהשער מעולם לא אימת $\Leftarrow$ מוקל על-ידי ה-`egress ACL` כגיבוי האמיתי, `re-gating` של הפניות, ו-`re-scoping` לכל תגלית בכתיבה.
- **שם-מארח in-scope הנפתר ל-IP פרטי/פנימי/metadata** (`SSRF`-אל-תשתית) $\Leftarrow$ מוקל על-ידי **סירוב-קשיח מוקדם ולא-ניתן-לתצורה** של טווחים פרטיים/פנימיים/metadata לכל פעולה, כולל פסיבית.
- **DNS rebinding / TOCTOU** בין בדיקת-השער לחיבור $\Leftarrow$ מוקל על-ידי `resolve-once-and-pin`; כלים שאי-אפשר להצמיד ל-`resolver` של הליבה מסורבים לשימוש אקטיבי.
- **httpx סווג בטעות כפסיבי** $\Leftarrow$ הוסר; `httpx` הוא אקטיבי. "פסיבי" משמעו אפס מגע ביעד, והסיווג הוא תכונת `manifest` סטטית, לעולם לא נגזרת מפלט ה-`LLM`.
- **מגביל-קצב פר-כלי** שמאפשר ל-N כלים לצבור מעבר לתקרה $\Leftarrow$ הוחלף במגביל **גלובלי-יחיד לתהליך**; בדיקת `CI` מוודאת שהקצב המצרפי לעולם אינו חורג מהתקרות.
- **Resume המשחזר פעולה מאושרת** ללא אישור טרי $\Leftarrow$ מוקל על-ידי `re-gating` ב-`resume` ושריפת ה-`nonce` החד-פעמי לפני ה-`spawn`.
- **תבניות nuclei מסוכנות / OOB** $\Leftarrow$ הדרות-תגים כפויות, `-no-interactsh`, `allowlist` חיובי של תגים כתקרה האמיתית, ודגלי `re-enable` אסורים.

6. לולאת הבקרה האגנטית

- **מצבים:** `INIT → RECON → ENUMERATION → DISCOVERY → TRIAGE → REPORTING → DONE`, בתוספת חוצי-מצבים `HALTED` (סירוב-קשיח של השער, דביק) ו-`AWAITING_APPROVAL` (השהיית `HITL`).

יחידת העבודה = פעולה אחת (לא שלב). כל פעולה היא פריט תור עמיד הנע דרך מצבים מתמידים: `PLANNED` → `SCOPE_CHECKED` → (`APPROVAL_PENDING` → `APPROVED`) → `RATE_WAIT` → `RUNNING` → `NORMALIZING` → `DONE`, עם מצבי-קצה `DENIED` / `FAILED-DEAD` / `SKIPPED`. מחזור פר-טיק: `PLAN (LLM)` → `PRE-GATE` → `APPROVE (human, active only)` → `EXECUTE (kernel)` → `OBSERVE (parse-store)` → `RE-PLAN`, עם checkpoint לאחר כל מעבר, לצורך יכולת-שחזור מלאה.

חלוקת LLM / קוד (נושאת-משקל)

ה-LLM מחליט (הסקה)	קוד דטרמיניסטי בבעלות (אפס LLM)
איזה כלי/יעד הבא; היפותזות; ניתוח פלט; קורלציה; הצעות dedup; נרטיב חומרה; פרוזת הדיווח	שיוך היקף; סיווג פסיבי/אקטיבי; אימות אסימון-אישור; הרכבת argv; אכיפת קצב; commit של מעבר-שלב; כל עובדת-סמכות נגזרת מחדש מהתצורה

- המסיק מציע; הליבה מכריעה. הטענות `is_active` והלא-הרסניות מה-LLM הן ייעוציות; ה-manifest והשער מכריעים.
- קידום השלב הוא דטרמיניסטי** – מונע על-ידי קריטריוני-היציאה של כל שלב, לעולם לא על-ידי ה-LLM המצהיר "סיימתי". בקשה אקטיבית המגיעה לפני אבן-הדרך של recon פסיבי – מסורבת.

7. שער ההיקף (כיצד עוטפים כל קריאה לכלי)

7.1 יירוט. לכל כלי יש manifest בבעלות הליבה המצהיר היכן שוכנים היעדים שלו (`target_params` עם `kind` סמנטי), סיווג, `forced_flags`, `forbidden_flags`, `allowlist` סגור `allowed_extra_flags` (ברירת מחדל ריק), והמפענח שלו. הסוכן מגיש שדות מובנים; הליבה מרכיבה את ה-argv כרשימת Python עבור `subprocess` עם `shell=False`. אין `shell`, לעולם.

7.2 חילוף יעדים (כוללני-יתר, כשל-סגור).

- קובצי רשימת-יעדים ו-stdin הם **נתונים**, לעולם לא נתיבים שהסוכן מספק – הליבה מאמתת כל שורה וכותבת בעצמה את קובץ ה-temp המאומת. כל נתיב `-l/-iL/-w` שהסוכן מספק ← מסורב.
- ה-manifests חייבים להצהיר במלואם על כל מקור-יעד. מקור לא-מוצהר ← סירוב. כלי לא-מוכר ← מסווג אקטיבי (כשל-סגור). token בלתי-ניתן-לפירוק במיקום-יעד ← `DENY_HALT`.

7.3 נרמול והתאמה (out-of-scope גובר). כל atom עובר lowercase, הסרת נקודה סופית, ← IDNA/UTS-46 punycode, נרמול NFC, ודחיית תוויות מעורבות-סקריפט/מבלבלות. כתובות URL נפרסות בקפדנות כאשר המארח נלקח מרכיב ה-authority **לאחר השלכת ה-userinfo** (`http://in-scope@evil.com/` ← `evil.com`). סדר ההכרעה פר-atom:

- out_of_scope תחילה** ← סירוב הבקשה כולה (גובר אפילו על תו-כללי in-scope).
- in_scope** – דומיין מדויק, תו-כללי בתווית-שמאלית על **גבולות דומיין בר-רישום (PSL)** (`*.example.com`) מתאים ל-`a.example.com`, `a.b.example.com`; לא ל-`apex` החשוף; לא ל-`example.com.evil.com`, או בתוך טווח CIDR של `ip_ranges`.

3. אף אחד מהם $\Leftarrow$ DENY_HALT $\Leftarrow$ UNVERIFIED.

7.4 מדיניות IP / DNS. טווחים פרטיים/פנימיים, loopback, link-local, metadata (RFC1918, CGNAT), 169.254.169.254, **מסורבים-קשיח לכל פעולה כולל פסיבית.** פותרים פעם אחת בליבה; מקפאים את קבוצת ה-IP לתוך טביעת-האצבע של הפעולה; מצמידים את החיבור. שרשראות CNAME נעקבות בזמן-השער (כל קפיצה out-of-scope $\Leftarrow$ DENY_HALT). הפניות מושבתות כברירת מחדל; מארח Location מוגש-מחדש לשער כפעולה חדשה.

7.5 שער דו-דרגתי.

- דרגה 1 (כל הפעולות): פסק-היקף מצרפי; כל atom out-of-scope/לא-מאומת $\Leftarrow$ DENY_HALT.
 - דרגה 2 (אקטיבי): בנוסף דורש `authorized == true` וגם `active_actions_allowed == true` וגם אסימון-אישור אנושי חד-פעמי תקף.
- ה-matcher מגיע עם **Hypothesis** + **קורפוס יריבי של ~50 מקרי "חובה-לסרב"** הרצים ב-CI כשער-מיזוג (merge gate).

8. אישור אנושי בלולאה

Passive-first נאכף מבנית על-ידי שלושה מנעולים עצמאיים: סיווג ה-manifest, שער PassiveFirstGate, (מארח זקוק ל- $1 \leq$ תצפית פסיבית מוצלחת לפני כל פעולה אקטיבית), ושער-ר-שלב (RECON מתיר רק כלים פסיביים).

ApprovalRequest (מה שהאדם רואה): הקשר התוכנית (name/platform/policy_url), הכלי, ה-argv, **המלא והפתור בית-אחר-בית** (כולל דגלי-קצב מוזרקים, תוכן קובץ-רשימה מורחב, IP-ים פתורים ומוצמדים), **ScopeDecision** פר-יעד, הסיווג, רדיוס-הנזק (מספר בקשות צפוי, concurrency, rps), הרציונל של ה-LLM + מזהי-ממצא מקוריים, out_of_scope.notes מוצג מילה-במילה, ו-proposal_fingerprint + expires_at קצר.

ApprovalGrant / אסימון – חד-פעמי ולא-מרחיב:

- כבול ל-, sha256(canonical(tool, sorted argv, sorted targets, is_active, phase, run_id)) המחושב על-ידי פונקציית-נרמול יחידה המשמשת בבניית-הבקשה ובאימות-ההרצה כאחד.
- שימוש-יחיד: ה-nonce נשרף במאגר לפני ה-spawn (קריסה אינה יכולה להריצו מחדש).
- לא-מרחיב: אדם רשאי רק לצמצם (למחוק יעדים, להנמיך קצב); הפעולה המצומצמת עוברת re-gating. לעולם לא להוסיף יעד, להעלות קצב, או להחליף כלי.
- מאומת-מחדש ב-resume מול ה-scope.yaml הנוכחי, עם בדיקה חוזרת של `authorized == true`.
- כשל-סגור: אין-TTY / פקיעה / EOF $\Leftarrow$ DENY. ריצות ללא-השגחה מסרבות לכל פעולה אקטיבית.

9. מאגר הממצאים ויומן הביקורת

- `SQLite` (`findings/<program-handle>/store.db` , WAL) = מקור-אמת בר-שאלתה.
 - **JSONL מוסף-בלבד, משורשר-hash ב-SHA-256** (`events.jsonl`) = מקור-אמת עמיד-לזיוף. ה-JSONL מוסף לפני שה-DB משתנה; ה-DB הוא מטמון הניתן לשחזור מהיומן.
 - **הרשאת כתיבה היא של הליבה בלבד** – קוד לא-מהימן לעולם אינו כותב שורות, כך שמודל עוין אינו יכול להזריק הצהרת "in-scope" מזויפת.
 - כל שורת `finding` , `tool_invocation` , `asset` ו- `scope_decision_id` נושאת `NOT NULL`.
- טבלאות ליבה (בקיצור):** `program` , `scope_snapshot` , `(raw_yaml_sha256 +)` , `run` , `scope_decision` ; `agent` , `skill` , `tool` , `tool_invocation` , `asset` ; פולימורפי (`kind ∈`) `identity_key` , `domain/subdomain/host/ip/port/service/url/endpoint` , `scope_status` , `severity+rank` , `finding` ; `asset_sighting` , `asset_edge` , `(probe_state , resolve_state` , `(confidence, status, triage_state , dedup_key` , `scope_decision_id` , `NOT NULL` , `evidence` , `finding_asset` (מצביע blob מכותב-תוכן, מצונזר כברירת מחדל); `approvals` , `llm_analyses` , `checkpoints` .
- שרשור השלבים הוא שאלות תור-עבודה, לא הודעות.** `recon` כותב `assets` ; `enum` בוחר תת-דומיינים in-scope ממתינים ← כותב `discovery` ; `services` ← ממצאים מועמדים; `triage` מבצע `dedup` ; `reporting` קורא מאושרים. **ההיקף מאומת-מחדש בקריאה** – הסרת דומיין באמצע ריצה מפסיקה מיידית להפוך את תת-הדומיינים שלו לברי-פעולה. ראיות מצונזרות כברירת מחדל (תקציר $\geq 1KB$); קובץ מאגר אחד לכל מעורבות, לבידוד הרשאה.

10. מצאי הכלים והערבויות הלא-הרסניות

הכלים מוצהרים ב- `config/tools.yaml` (מנוהל-גרסאות, נסקר-קוד – זו מדיניות בטיחות). הסיווג הוא סמכותי ב-`manifest`. **המגנים נושאי-המשקל** הם (1) `argv` הנבנה מ-`allowlist` הסגור `allowed_extra_flags` , (2) `forbidden_flags` , ו-(3) ה-`egress ACL`. דגל `-x=false` כפוי הוא הגנה-לעומק בלבד; ה-**CI מאמת כל דגל forced/forbidden מול גרסת הכלי המוצמדת** כך ששדרוג גרסה לא יכול להפוך ערובה ל-`no-op` בשקט.

כלי	מחלקה	שלב	כפוי (הגנה-לעומק)	אסור
subfinder	passive	Recon	מקורות פסיביים בלבד (<code>-silent</code>)	<code>-all</code> , <code>resolution</code> , <code>אקטיבי</code>
amass	passive	Recon	<code>-passive</code>	<code>-active</code> , <code>-brute</code> , <code>-ip</code>
gau	passive	Recon	שליפת ארכיון בלבד (<code>--subs=false</code>)	מצבי <code>live-probe</code>
httpx	active	Enum	קצב מה- <code>broker</code> ; הפניות לא-מופעלות	<code>-fhr /fr</code> , <code>-follow-redirects</code> , <code>-/</code>
naabu	active	Enum	<code>connect-scan</code> , קצב מה- <code>broker</code> , פורטים חסומים	<code>-p-</code> (טווח-מלא), <code>-s</code> (SYN)

out-of-scope / redirect- דגלי- follow	עומק-חסום, מוגבל-להיקף, קצב מה-broker	Enum	active	katana
-include-tags/-fuzz/-dast/-payload/-code/- LLM; headless תגי- מוצטלים עם allowlist לאחר הדורות	-exclude-tags dos, intrusive, fuzzing, brute- , -no-interactsh , force, rce, sqli קצב מה-broker , dast=false	Discovery	active	nuclei
- recursion* , עקיפת -t /rate	broker, -rate , -recursion=false wordlist מיון	Discovery	active	ffuf

egress-ו OAST/interactsh שרירותי לצד-שלישי כבויים כברירת מחדל; כל ה-DNS של כלים אקטיביים נכפה דרך ה-resolver של הליבה. ffuf / naabu / nuclei רצים אך ורק דרך הפרופיל הבטוח הקנוני היחיד שלהם - אף רכיב אינו רשאי להריצם ישירות.

11. קטלוג ה-Skills

Skill הוא יחידת-התזמור הקטנה ביותר. כל Skill עוטף כלי אחד בדיוק ואינו רשאי לפרוש על-פני + passive active (זו הסיבה שהזחילה מפוצלת ל- gau-archive (פסיבי) ו- crawl (אקטיבי, katana)). תת-סוכן פולט כוונת Skill (intent); הליבה מאמתת אותה מול קבוצת ה-Skills המותרים של הסוכן, מריצה כל יעד מחדש דרך שער- ההיקף, ורק אז מבצעת spawn. ה-LLM לעולם אינו קו-ההגנה האחרון.

Skill	שלב	מחלקה	עוטף	דורש אישור?
subdomain-enum	recon	passive	subfinder	לא
amass-passive	recon	passive	amass	לא
gau-archive	recon	passive	gau	לא
liveness-probe	enumeration	active	httpx	כן
port-enum	enumeration	active	naabu	כן
crawl	enumeration	active	katana	כן
vuln-scan	discovery	active	nuclei	כן
fuzzing	discovery	active	ffuf	כן
triage	triage	none	-	לא (re-probe הוא פעולה נפרדת ומשוערת)
report-gen	reporting	none	-	לא (אין מגע; לעולם לא משדר)

כל מפרט Skill מצהיר: מטרה, קלט/פלט מטופסים (קריאות/כתיבות למאגר), הכלי הנעטף, בלוק authorization (requires_passive_baseline / requires_active_approval / requires_authorized), עקרונות

ה- `scope_check`, הערבויות הלא-הרסניות שלו, ובדיקות הרצות ללא יעד חי (חובה-לסרב `out-of-scope`, דגלי `forced/forbidden`, ואיסור-ללא-אישור).

מחזור החיים של `asset` (אוצר-המילים המשותף להעברה): `scope_status ∈ {in_scope, out_of_scope, unresolved, unresolvable}` (מחושב `out-of-scope` בתחילה בכתובה); `resolve_state ∈ {pending, resolved, unresolvable}` (נקבע על-ידי הליבה בשלב `resolve-and-pin`, המממש גם `ip / kind=host`); `probe_state ∈ {unprobed, live, dead}` (נקבע על-ידי `liveness-probe`). ה-**בסיס הפסיבי** (**passive baseline**) הוא שורת `assets` `in-scope` אשר `source ∈ {subfinder, amass, gau}` `archive`.

12. תת-הסוכנים וחוזת ההעברה

חמישה תת-סוכנים ממופים 1:1 לשלבים. כל אחד הוא יחידת-הסקה תחומה של LLM הקוראת `FindingsStoreView` מתוחם וקריאה-בלבד, מתכנתת כוונות-Skill, וכותבת ממצאים מנורמלים - **אף אחד אינו יכול לבצע דבר**. ההעברה היא **סטיגמרגית**: סוכן משאיר שורות עם `status`; סוכן השלב הבא בוחר אותן כתור-עבודה. אין ערוץ סוכן-לסוכן.

סוכן	שלב	מחלקה	רשאי לקרוא ל-	צורך ← מייצר (דרך המאגר)
recon	RECON	passive	subdomain-enum, amass-passive, gau-archive	scope seeds → assets(status=new, resolve_state=pending)
kernel resolve+pin	בין שלבים	-	-	resolve_state=resolved, materialize host/ip
enumeration	ENUMERATION	active	liveness-probe, port-enum, crawl	assets(new,resolved) → assets(enriched, probe_state=live dead)
vuln-discovery	DISCOVERY	active	vuln-scan, fuzzing	assets(enriched, live) → findings(candidate)
triage	TRIAGE	none	triage	findings(candidate) → findings(validated dismissed duplicate)
reporting	REPORTING	none	report-gen	findings(validated) → local report file

- קריטריוני-יציאה דטרמיניסטיים** מקדמים כל שלב (ה-FSM, לעולם לא ה-LLM). לדוגמה: recon עוצר כאשר שני מעברים פסיביים רצופים אינם מניבים `assets` חדשים `in-scope` (עם תקרת-מעברים קשיחה); enumeration עוצר כאשר כל `asset in-scope` עבר `probe` או הוא `unresolvable`, כולל `endpoints` חדשים שנחלו; triage עוצר כאשר לא נותר מועמד וכל `re-probe` שהועלה הגיע למצב-קצה (אושר+רץ, נדחה/פקע, או נמשך).
- סוכנים אקטיביים** (`enumeration`, `vuln-discovery`) דורשים `authorized==true` + אישור חד-פעמי פר-פעולה + `egress` מאומת + בסיס פסיבי, ללא איגוד (batching) או הרחבה.

• **Triage** אינו נוגע ביעד כלל; re-probe מאשש שהוא מעלה נשלח כפעולה אקטיבית נפרדת, קריאה-בלבד, המאושרת בדרג גבוה יותר. **Reporting לעולם אינו משדר** - הוא מייצר קובץ מקומי; ההגשה נותרת פעולה אנושית מכוונת.

• **ב-Resume**, כוונה אקטיבית מאושרת-אך-לא-בוצעה עוברת re-gating מול ה- `scope.yaml` הנוכחי ובדיקה חוזרת של `authorized==true`; ה-`nonce` החד-פעמי נשרף לפני ה-`spawn`, כך שלעולם אינו משוחזר.

13. מחסנית טכנולוגית

נושא	בחירה	נימוק
שפה	Python 3.11+, ruff + mypy (strict on scope/,tools/,kernel/) + pytest	הצעות/החלטות מטופסות הן בקרת-בטיחות
מנוע הסוכן	Custom asyncio FSM	מחזור-החיים הוא מכונת-מצבים; שער/אישור כקוד-על-הקצוות
LLM	לפלט מובנה, מאחורי tool-use anthropic SDK, Reasoner port	הסקה-בלבד; פלט-סכימה ← הצעות מאומתות-pydantic
תצורה/מודלים	pydantic v2 + yaml.safe_load	פירוק מטופס כשל-סגור; <code>extra='forbid'</code> ; <code>scope_config_hash</code>
מאגר ממצאים	stdlib sqlite3 (WAL)	אפס-תשתית, joins, ACID, ל-chaining/dedup
יומן ביקורת	JSONL, append-only, SHA-256 hash-chained	עמיד-לזיוף, בר-grep, בר-שחזור
הגבלת קצב	גלובלי יחיד, token bucket + asyncio.Semaphore	נקודת-האכיפה היחידה לתקרות מצרפיות
תת-תהליך	asyncio.create_subprocess_exec, shell=False, setrlimit, timeouts	רשימות argv בלבד; אין injection
Sandbox	netns + nftables (Linux) / enforced allowlisting proxy (macOS)	egress ACL חובה - הקיר השני
Matcher	custom + Public Suffix List + Hypothesis	גבולות דומיין בר-רישום; קורפוס יריבי ב-CI
HITL / CLI	typer + rich	הנחיית-אישור מפורטת המציגה argv מדויק + diff היקף

14. סכימת התצורה

```
program{name, config/scope.yaml (git-ignored, לכל-מעורבות) - מקור-הסמכות היחיד. שדות:
,authorized (bool), platform, policy_url, authorized_until, last_verified_at
,scope_semantics{ip_ranges_mode}, in_scope{domains, subdomains, ip_ranges}
,out_of_scope{domains, subdomains, ip_ranges, notes}
```

, (bool) `active_actions_allowed`, `rate_limits{requests_per_second, max_concurrency}`
. `notes`

תוספות שאומצו בסשן זה (תואמות-לאחור, ברירת-מחדל קפדנית/בטוחה):

- `last_verified_at` / `program.authorized_until` - השער עוצר-אוטומטית (auto-HALT) על תוכנית מיושנת/פקועה.
- `scope_semantics.ip_ranges_mode: boundary` - `ip_ranges` הוא גבול קשיח (שם in-scope חייב להיפתר גם לתוך טווח in-scope), לא רק קבוצת-היתר נוספת.
- `config/tools.yaml` (מנוהל-גרסאות) - manifests הצהרתיים של כלים: סיווג, `target_params`, `forced_flags`, `forbidden_flags`, `allowed_extra_flags`, `rlimits`, `timeouts`. מנוהל-גרסאות ונסקר-קוד כי הוא *מדיניות בטיחות*, נפרד מ-`scope.yaml` הסציפי-ליעד.
- שני תבניות ה-`example.yaml` מסופקות; תצורה אמיתית חייבת להיבדל מהתבנית ב-`content hash`, אחרת המנוע מסרב לרוץ.

15. החלטות שנעלו ופסיקות

1. `ip_ranges` הוא **גבול קשיח**; out-of-scope גובר על-פני שני הממדים - השם וה-IP.
2. `*.example.com` מוציא את ה-**apex החשוף** `example.com` (יש לרשום תחת `domains`); תווים-כלליים מתאימים על גבולות דומיין בר-רישום (PSL).
3. **כלים שאי-אפשר להצמיד מסורבים לשימוש אקטיבי** (מוגבלים ל-egress-ACL + passive).
4. **מגן סירוב-להרצה**: המנוע מסרב-קשיח אם `scope.yaml` תואם את התבנית המסופקת ב-`content-hash`. `sentinel`
5. **עצירה-אוטומטית על מיושנות** דרך `last_verified_at` / `authorized_until`.
6. `config/tools.yaml` הוא מדיניות בטיחות **מנוהלת-גרסאות**.
7. **http הוא אקטיבי**; "פסיבי" משמעו אפס מגע ביעד; הסיווג הוא תכונת manifest סטטית, לעולם לא נגזרת מפלט LLM.
8. **מגביל-קצב גלובלי-יחיד לתהליך**; מגבילים פר-כלי אסורים.
9. קטלוג ה-Skills הורחב מ-7 ל-10 - כלל הכלי-האחד פיצל את `crawl-and-endpoints` ל-`gau-archive` + `crawl`, ונוספו `amass-passive` + `port-enum`.

16. שאלות פתוחות וסיכונים שיוריים

מסומנות להכרעה לפני / במהלך המימוש:

- **הדרות out_of_scope מובנות** - עד לאימוץ, הערות בשפה טבעית (למשל "אין endpoints של צד-שלישי לתשלומים") מוצגות מילה-במילה בכל אישור + בדיקה מקדימה ייעוצית-בלבד של ה-LLM; הן אינן נאכפות

מכנית.

- **dns_resolution_policy** - האם שם in-scope הנפתר ל-IP לא-רשום (לא-פרטי) נדחה-אוטומטית או נשאל (IP-ים פרטיים/metadata תמיד מסורבים-קשיח).
- **שלמות שרשרת-אספקה** - בינארי כלי מודבק/מתעדכן-אוטומטית נתפס רק על-ידי ה-egress ACL; נדרש בינארים מוצמדים + checksums וכיבוי עדכון-אוטומטי.
- **מפתח חתימת אסימון-האישור** (HMAC) - אחסון וסבב; נדרש תכן מבוסס OS-keychain.
- **קטיעת זנב יומן-הביקורת** - שרשרת-hash פשוטה אינה מזהה קטיעה מלאה של הזנב; יש לחתום על ראש-השרשרת או לעגן מחוץ-לתיבה.
- **מתג-הרג גלובלי / עצירת-חירום** לביטול מיידי של כל ה-ALLOW-ים והאסימונים הלא-שרופים באמצע ריצה.
- **ערוץ ממשק-האישור** (הנחיית CLI מול file-drop מול webhook) וסמנטיקת ה-resume שלו.
- **הגברת OSINT צד-שלישי** - הבטחה שמקורות פסיביים לא יהפכו לוקטור הגברה.

17. מפת דרכים למימוש

הסדר הנכון בונה תחילה את הרכיבים נושאי-המשקל וחסרי-ה-LLM, ובודק אותם יריבית, לפני שדבר יכול לגעת ברשת.

1. **Matcher + Validator של שער-ההיקף בליבה** - מודלי pydantic ל- `tools.yaml` / `scope.yaml`; נרמול; סדר-ההכרעה (out-of-scope-תחילה, CIDR, PSL/wildcard); בדיקות-תכונה של Hypothesis + קורפוס ~50-המקרים "חובה-לסרב" כשער-מיזוג ב-CI. (אין רשת, אין LLM).
2. **מאגר ממצאים + יומן-ביקורת משורשר-hash** - כותבי-ליבה בלבד; סכימה מ-9§; rebuild-from-log.
3. **מגבר-קצב + token-bucket+semaphore - sandbox/egress** גלובלי יחיד; egress ACL הנוצר מ- `scope.yaml`; בדיקת CI המוודאת egress-allows == policy-allows וקצב מצרפי ≥ תקרות.
4. **בסיס ToolWrapper + מפרטי כלים** - מחזור-החיים הקבוע עם שער-ההיקף המנדטורי; dry-run.
5. **ברוקר-אישורים** - האסימון החד-פעמי הכבול-לטביעת-אצבע; התנהגות כשל-סגור לא-אינטראקטיבית.
6. **Orchestrator FSM** - מכונת-מצבי-השלב, re-gating, checkpointing ב-resume.
7. **Reasoner port + תת-סוכנים + Skills** - שכבת ה-LLM הלא-מהימנה, אחרונה, על-גבי ליבה מוכחת.

אבן-הדרך 1 היא בעלת המנוף הגבוה ביותר: כל השאר בוטח ב-matcher. הוא חייב להיות נכון לפני שמנה בודדת אי-פעם אפשרית.

נספח: מילון מונחים (Glossary)

אנגלית	עברית / הסבר
scope-gate	שער ההיקף - נקודת-החנק המאמתת כל פעולה מול <code>scope.yaml</code>

trusted kernel	הליבה המהימנה - הרכיב היחיד המחזיק subprocess/socket/DB-write
fail-closed	כשל-סגור - ספק/עמימות/שגיאה מובילים לסירוב, לעולם לא להיתר
passive / active	פסיבי (אפס מגע ביעד) / אקטיבי (מנה לתשתית היעד)
egress ACL	רשימת בקרת יציאה רשתית - הקיר השני, נוצר מ-scope.yaml
provenance	שרשרת-מקור - עקיבות מלאה מכל ממצא אל הריצה שיצרה אותו
stigmergic hand-off	העברה סטיגמרגית - מסירה דרך המאגר (status כתור-עבודה), לא הודעות
resolve-once-and-pin	פתירה-פעם-והצמדה - הקפאת ה-IP למניעת (TOCTOU) DNS rebinding
wildcard	תו-כללי - התאמת *.example.com על גבולות PSL
approval token / grant	אסימון אישור - חד-פעמי, לא-מרחיב, כבול לטביעת-אצבע של הפקודה
passive baseline	בסיס פסיבי - תצפית פסיבית נדרשת למארח לפני כל פעולה אקטיבית
findings store	מאגר הממצאים - מקור-האמת ב-SQLite המשרשר תוצאות בין שלבים
responsible disclosure	גילוי אחראי - מסירת הממצאים לבעל התוכנית
Skill / sub-agent	יחידת יכולת עוטפת-כלי / יחידת-הסקה תחומה בבעלות שלב

הערה: שמות קבצים, שדות תצורה, שמות כלים (subfinder/httpx/nuclei וכו'), דגלי CLI, ומזהי מצב/סטטוס נשמרו באנגלית ובכיווניות LTR בתוך הפריסה ה-RTL, כמקובל בתיעוד טכני.